

Answer Set Programming goes to School

Can High School Students Learn Declarative Programming?

Racquel Dennison¹ Emmanuelle Dietz² Hanna Philipp³

¹University of Cape Town and CAIR, South Africa

²Airbus Central Research & Technology, Hamburg, Germany

³Weißeritzgymnasium Freital, Freital, Germany

July 16, 2026

Research Question

Is ASP suitable for learning at high school level?

Human reasoning is **non-monotonic** [1]

⇒ Classical logic inadequate

⇒ ASP rooted in non-monotonic logics [2, 3]

Can a high school student learn and apply ASP?

Approach: A high school student solves their school's classroom scheduling problem

Why ASP?

- Declarative thinking
- Proven for scheduling [4, 5]
- But is it *learnable*?

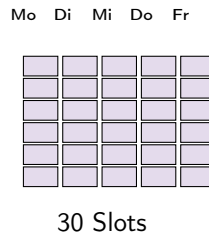
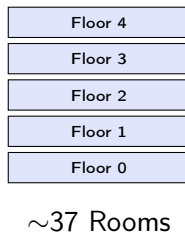
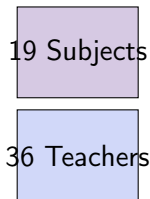
Why This Matters?

- Different paradigm
- Real-world problems
- Early CS education

The School Scheduling Problem

Weißeritzgymnasium Freital

Assign: Classes $\rightarrow$ Rooms + Teachers + Time slots

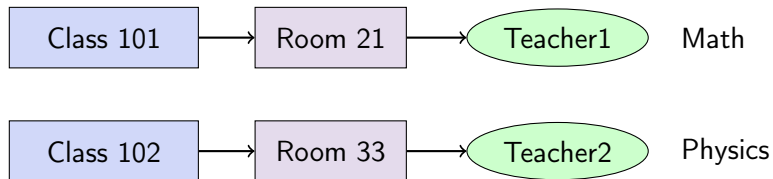


Constraints: No double-booking • Specialized rooms • Block hours (max 2 consecutive)

Optimize: Minimize distances • Maximize blocks • Teacher work-life balance

Example: One Time Slot

Monday, 10:00 AM (Slot 3)



Question: How to encode this in ASP?

ASP Encoding: Facts

Student Feedback: "Writing facts is easy"

Ground truth - what we know:

```
% Teachers and their qualifications
teacher(teacher1, ma).
teacher(teacher1, info).
teacher(teacher2, ph).

% Rooms with coordinates (ID, X, Y, Floor)
room(21, 0, 0, 2).
room(33, 2, 0, 3).

% Curriculum: Class needs X hours of subject F
curriculum(101, ma, 4).
curriculum(102, ph, 2).

% Specialized rooms
room_subject(33, ph).      % Room 33 for physics only
```

ASP Encoding: Standard Rules

Student Feedback: "Intuitive once an example is provided"

Derive new information from facts:

```
% Calculate distance between rooms (Manhattan + floor penalty)
dist_x(R,R1,DX) :- room(R,X,_,_), room(R1,X1,_,_),
                  X >= X1, DX = X - X1.

dist(R,R1,D) :- dist_x(R,R1,DX), dist_y(R,R1,DY),
               dist_z(R,R1,DZ),
               D = DX + DY + DZ*4.      % Floor changes cost 4x
```

Example: Room 21 (0,0,2) to Room 33 (2,0,3)

$$\text{Distance} = |0 - 2| + |0 - 0| + |2 - 3| \times 4 = 2 + 0 + 4 = \mathbf{6 \text{ units}}$$

Learning: Initial encoding didn't capture all edge cases

ASP Encoding: Choice Rules

Student Feedback: "More difficult - managing bounds is tricky"

Let the solver decide - generate possible schedules:

```
2{allocation(K,R,F,T,S,L): teacher(L,F),  
                        room_subject(R,F),  
                        week(T,S)}2  
  
:- curriculum(K, F, X), X=2.
```

Meaning: Choose exactly **2 allocations** for class K and subject F when curriculum requires 2 hours

Example result:

```
allocation(102, 33, ph, mo, 3, teacher2).    % Monday slot 3  
allocation(102, 33, ph, mi, 2, teacher2).    % Wednesday slot 2
```

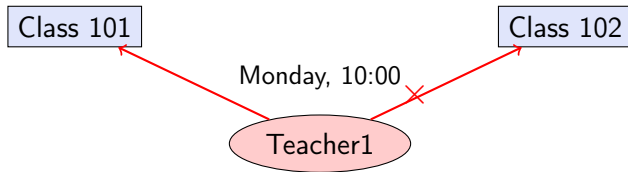
Challenge: Understanding min/max bounds and rule head structure

ASP Encoding: Integrity Constraints

Student Feedback: "More accessible than choice rules"

Eliminate invalid solutions:

```
% Teacher cannot teach two classes simultaneously  
:- allocation(K1, _, _, T, S, L),  
   allocation(K2, _, _, T, S, L),  
   K1 != K2.
```



Why easier? Behavior similar to standard rules - just negation!

ASP Encoding: Aggregates

Student Feedback: "Initially confusing - which variables?"

Count or sum over multiple solutions:

```
% Teacher should have at least 1 day off (home office)
:- teacher(L, F),
   #count{T, S : allocation(K, R, F, T, S, L)} > 4.
```

Common mistake: Forgot grounding predicate teacher(L, F)!

```
% Calculate total travel distance
travel_cost(K, D, T, S) :-
    allocation(K, R1, _, T, S1, _),
    allocation(K, R, _, T, S, _),
    dist(R1, R, D), S1 = S+1.

total_distance(K, G) :-
    #sum{D,T,S : travel_cost(K,D,T,S)} = G, class(K).
```

ASP Encoding: Optimization

Student Feedback: "Simpler than aggregates"

Multi-objective optimization:

```
% Minimize student travel
#minimize{G,K : total_distance(K,G)}.

% Maximize teaching blocks (consecutive hours)
block(F, S, S1, T, K, L) :-
    allocation(K, R, F, T, S, L),
    allocation(K, R, F, T, S1, L),
    S1 = S+1, S != 6.

sum_blocks(B) :- #sum{S,F,S1,T,K,L : block(F,S,S1,T,K,L)} = B.
#maximize{B : sum_blocks(B)}.
```

Conflicting objectives → Pareto front analysis

Fewer variable constructs make optimization statements easier to understand

Learning Experience Summary

What was EASY:

- ✓ Writing facts
- ✓ Standard rules (with examples)
- ✓ Integrity constraints
- ✓ Immediate solver feedback

What was HARD:

- ! Choice rule bounds
- ! Aggregates (variables)
- ! Covering edge cases
- ! Fact structure design

Ease



Facts

Rules

Constraints

Choice

Optimize

Aggregates

Execution:

- Grounding: minutes
- Solving: varies
- Optimal found: ✓

Output:

- Full schedule (CSV)
- Distance metrics
- Multi-objective costs

Result: Generated schedules are *actually usable* by the school!

System: Python + Clingo + Pandas

ASP for Scheduling:

System	Domain	Reference
teaspoon	University timetabling	[4]
LUNCH	Course scheduling	[5]
Nurse scheduling	Hospital shifts	[6]
Chemotherapy	Treatment planning	[7]

Our Contribution:

First study evaluating ASP from a **high school student's learning perspective**, not just as a solving tool.

Can high school students learn ASP?

YES! (with caveats)

- ① **Declarative thinking is learnable**
 - Clear separation: facts vs. decisions
 - Logical structure is intuitive
- ② **Learning curve exists**
 - Facts, rules, constraints: accessible
 - Aggregates: challenging (needs careful teaching)
- ③ **Examples + feedback = key to success**
 - Iterative development crucial
 - Real problems motivate learning

Questions?

Thank you for your attention!

Racquel Dennison, Emmanuelle Dietz, Hanna Philipp

Answer Set Programming goes to School

References I



R. M. J. Byrne.

Suppressing valid inferences with conditionals.

Journal of Memory and Language, 31:61–83, 1989.



G. Brewka, T. Eiter, and M. Truszczyński.

Answer set programming at a glance.

Communications of the ACM, 54(12):92–103, 2011.



M. Gebser, B. Kaufmann, and T. Schaub.

Conflict-Driven Answer Set Solving: From Theory to Practice.

Artificial Intelligence, 187–188, 2012.



M. Banbara et al.

teaspoon: solving the curriculum-based course timetabling problems with ASP.

Annals of Operations Research, 275(1):3–37, 2019.



A. Lima et al.

LUNCH: an Answer Set Programming System for Course Scheduling.

ENIAC, 2023.



H. Nabeshima, M. Banbara, T. Schaub, and T. Soh.

The ASP-based Nurse Scheduling System.

EPTCS, 439:334–348, 2026.



C. Dodaro et al.

An ASP-based Solution to Chemotherapy Treatment Scheduling.

TPLP, 21(6):835–851, 2021.